

CSA 1507 CLOUD COMPUTING AND BIG DATA ANALYSIS

- 1) Create a simple cloud software application for Flight Reservation System using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as one way, round trip, From, to, Departure, Departure, Travelers & Class, Fare Type, International and Domestic Flights etc.

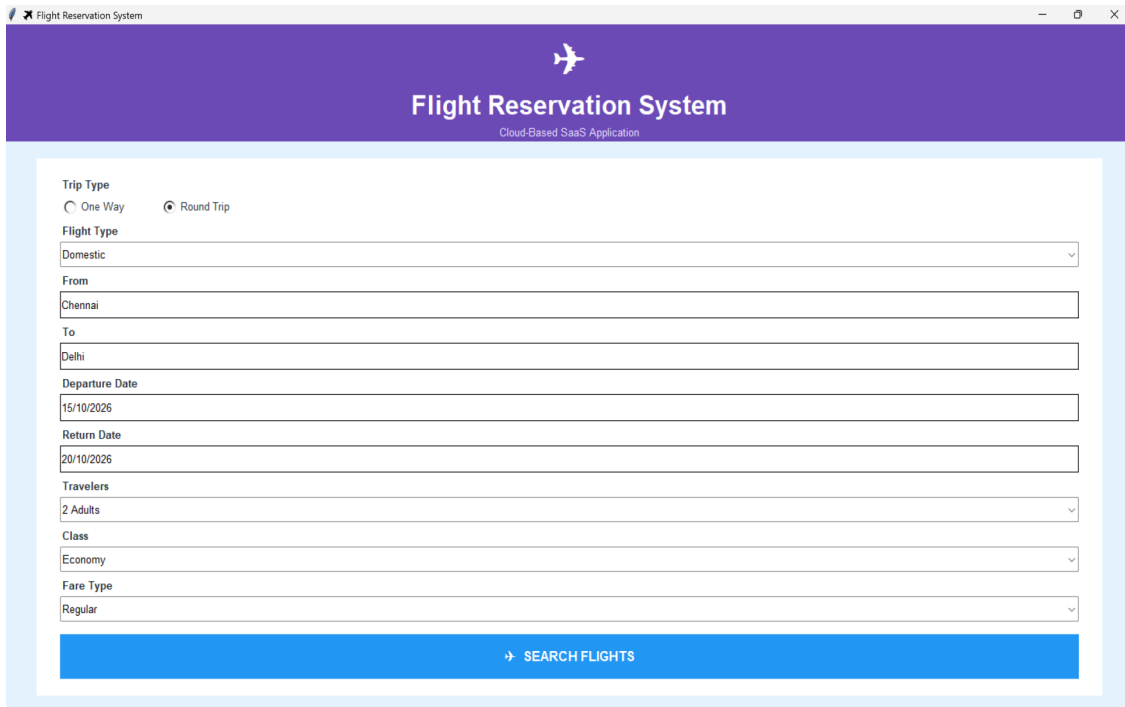
AIM

To create a simple **Flight Reservation System** using Python that demonstrates a cloud-based **SaaS application** with options for domestic and international flights, trip type, travel details, class, and fare type.

PROCEDURE

1. Open **Python IDLE** and create a new Python file.
2. Write the Python program using the **Tkinter** library.
3. Design the Flight Reservation System with fields for:
 - One Way / Round Trip
 - Domestic / International
 - From and To
 - Departure and Return Date
 - Travelers
 - Class
 - Fare Type
4. Add a **Search Flights** button to collect the entered details.
5. Save the program as flight_reservation.py.
6. Run the program using **F5** in Python IDLE.
7. Enter the flight details and click **Search Flights**.
8. Display the entered reservation details in a confirmation window.

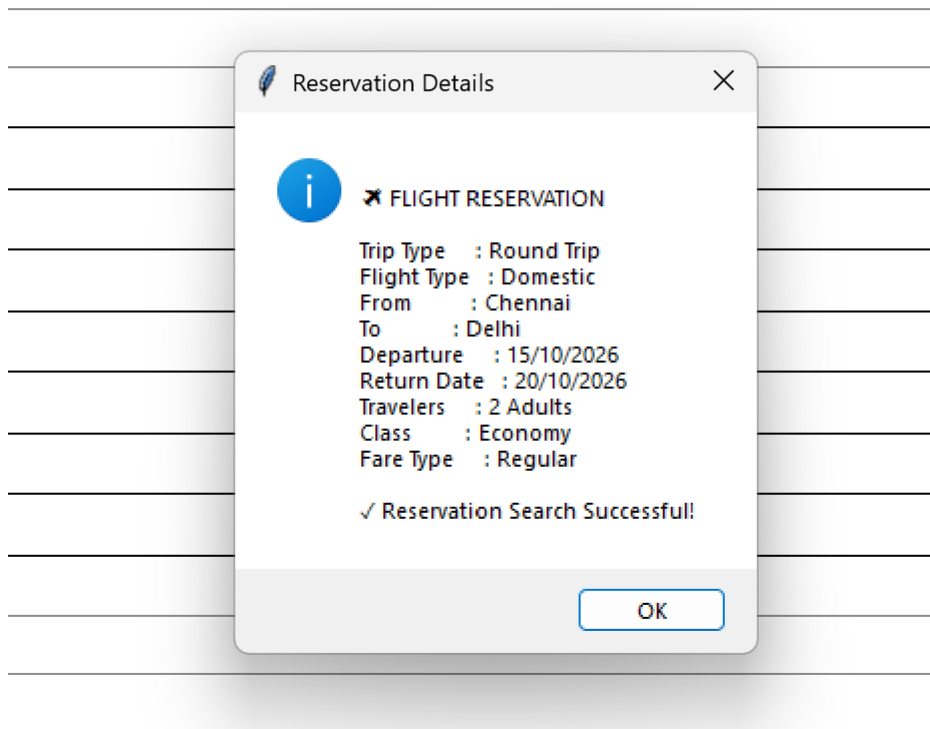
INPUT



The screenshot shows a web application titled "Flight Reservation System" with the subtitle "Cloud-Based SaaS Application". The form includes the following fields and options:

- Trip Type:** Radio buttons for "One Way" and "Round Trip" (selected).
- Flight Type:** A dropdown menu with "Domestic" selected.
- From:** A text input field with "Chennai" entered.
- To:** A text input field with "Delhi" entered.
- Departure Date:** A date input field with "15/10/2026" entered.
- Return Date:** A date input field with "20/10/2026" entered.
- Travelers:** A text input field with "2 Adults" entered.
- Class:** A dropdown menu with "Economy" selected.
- Fare Type:** A dropdown menu with "Regular" selected.
- Search Button:** A blue button labeled "SEARCH FLIGHTS" with a right-pointing arrow.

OUTPUT



RESULT

The **Flight Reservation System** was successfully created and executed using Python. The application accepts flight and passenger details and displays the reservation information successfully.

2. Create a simple cloud software application for Property Buying & Rental process (In Chennai city) using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as Buy, Rent, commercial, Rental Agreement , Rental Agreement, Area, place, Range, property value etc.

AIM

To create a simple **cloud-based Property Buying and Rental System for Chennai city** using Python, demonstrating the **SaaS model** with options for buying, renting, residential and commercial properties.

PROCEDURE

1. Open **Python IDLE** and create a new Python file.
2. Write the Python program using the **Tkinter** library.
3. Create fields for:
 - Buy / Rent
 - Residential / Commercial
 - Rental Agreement
 - Area
 - Place in Chennai
 - Price Range
 - Property Value
4. Add a **Search Property** button.
5. Save the program as `property_chennai.py`.
6. Run the program using **F5**.
7. Enter or select the required property details.
8. Click **Search Property**.
9. Display the entered property details as the search result.

INPUT

Chennai Property Buying & Rental System



Chennai Property Buying & Rental

Cloud-Based SaaS Application

Purpose

Buy

Property Type

Residential

Rental Agreement

Not Required

Area (sq.ft)

1200

Place in Chennai

Anna Nagar

Price Range

₹20 Lakhs - ₹50 Lakhs

Property Value

4500000

 SEARCH PROPERTY

Chennai Property Portal • Cloud-Based SaaS

OUTPUT

 Property Details



PROPERTY SEARCH RESULT

Purpose : Buy

Property Type : Residential

Rental Agreement : Not Required

Area : 1200 sq.ft

Place : Anna Nagar

Price Range : ₹20 Lakhs - ₹50 Lakhs

Property Value : ₹4500000

✓ Property Search Successful!

OK

 SEARCH PROPERTY

RESULT

The **Chennai Property Buying and Rental System** was successfully created and executed. The application accepts property details such as buying/renting option, property type, area, location, rental agreement, price range, and property value, and displays the property search result successfully.

3. Create a simple cloud software application for Car Booking Reservation System using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as search for cabs, from, to, rental , out station, Package type, hours/Days etc.

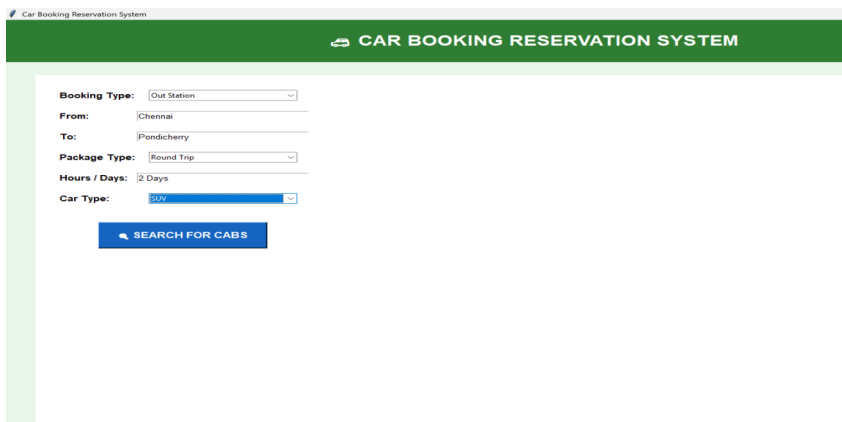
AIM

To create a simple **Car Booking Reservation System** using Python to demonstrate **SaaS (Software as a Service)** with features such as searching cabs, From, To, Rental, Out Station, Package Type, and Hours/Days.

PROCEDURE

1. Open **Python IDLE** and create a new Python file.
2. Import the **Tkinter** library to create the application interface.
3. Create fields for **Booking Type, From, To, Package Type, Hours/Days, and Car Type**.
4. Add a **Search for Cabs** button to process the booking details.
5. Save the program as `car_booking.py` and run it.
6. Enter the required details and click **Search for Cabs** to display the booking result.

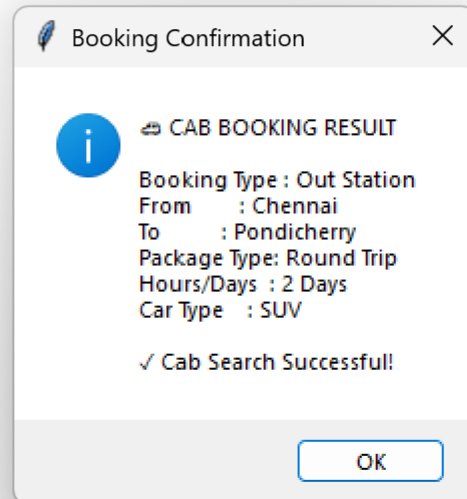
INPUT



The screenshot shows a web application titled "CAR BOOKING RESERVATION SYSTEM". The interface includes the following fields and controls:

- Booking Type:** A dropdown menu with "Out Station" selected.
- From:** A text input field containing "Chennai".
- To:** A text input field containing "Pondicherry".
- Package Type:** A dropdown menu with "Round Trip" selected.
- Hours / Days:** A text input field containing "2 Days".
- Car Type:** A dropdown menu with "SUV" selected.
- SEARCH FOR CABS:** A blue button with a magnifying glass icon.

OUTPUT



RESULT

The **Car Booking Reservation System** was successfully created and executed. The application accepts cab booking details and displays the **booking/search result** successfully.

4. Create a simple cloud software application for Library book reservation system for SIMATS library using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as Title of the book, author, publications, Year, edition, no of copies, Rack no, status (taken/returned) etc.

AIM

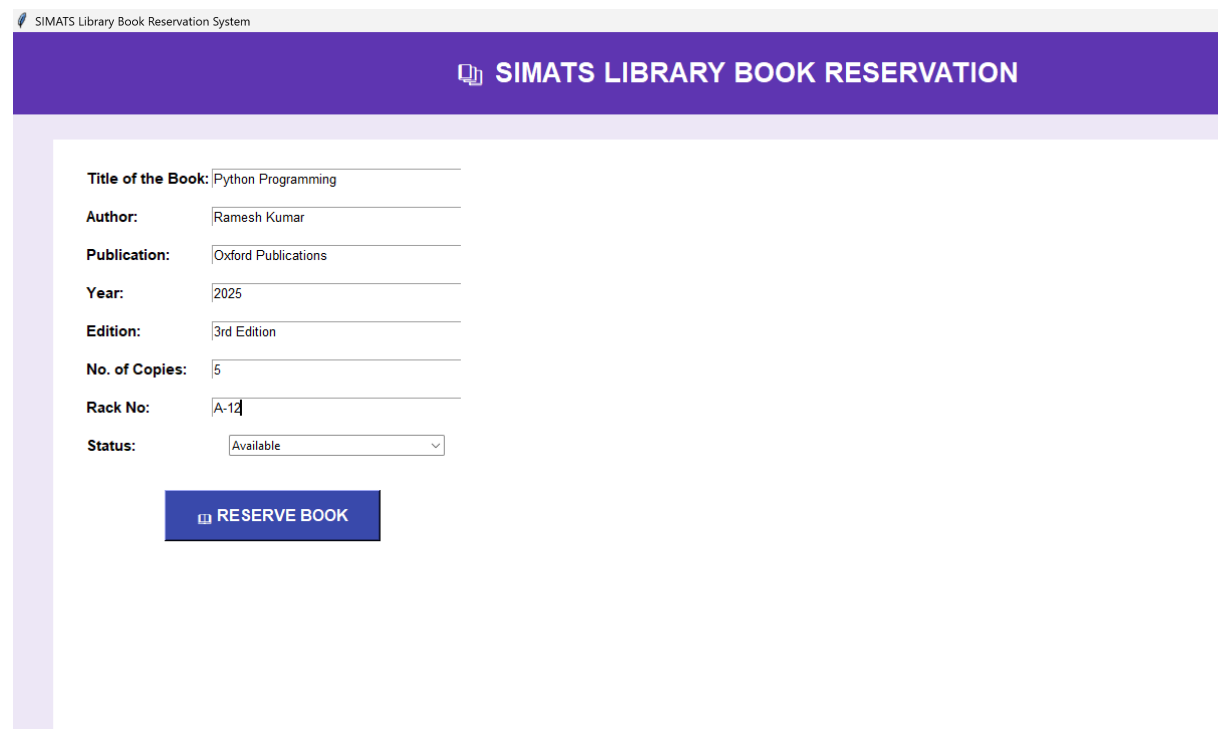
To create a simple **Library Book Reservation System for SIMATS Library** using Python to demonstrate **SaaS (Software as a Service)** with details such as book title, author, publication, year, edition, number of copies, rack number, and book status.

PROCEDURE

1. Open **Python IDLE** and create a new Python file.
2. Import the **Tkinter** library to create the application interface.

3. Create fields for **Title, Author, Publication, Year, Edition, No. of Copies, Rack No., and Status.**
4. Add a **Reserve Book** button to process the book details.
5. Save the program as library_reservation.py and run it.
6. Enter the book details and click **Reserve Book** to display the reservation result.

INPUT

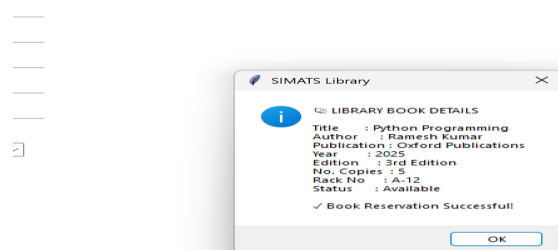


The screenshot shows the SIMATS Library Book Reservation System interface. It features a purple header bar with the text "SIMATS LIBRARY BOOK RESERVATION". Below the header, there is a form with the following fields:

- Title of the Book:** Python Programming
- Author:** Ramesh Kumar
- Publication:** Oxford Publications
- Year:** 2025
- Edition:** 3rd Edition
- No. of Copies:** 5
- Rack No:** A-12
- Status:** Available (dropdown menu)

Below the form is a blue button labeled "RESERVE BOOK".

OUTPUT



RESULT

The **SIMATS Library Book Reservation System** was successfully created and executed. The application accepts book details and displays the **library book reservation information** successfully.

5. Create a simple cloud software application for Mark Sheet Processing System using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as name of the student, Reg no of the student , Subjects , Marks, Average, percentage, rank, overall performance etc.

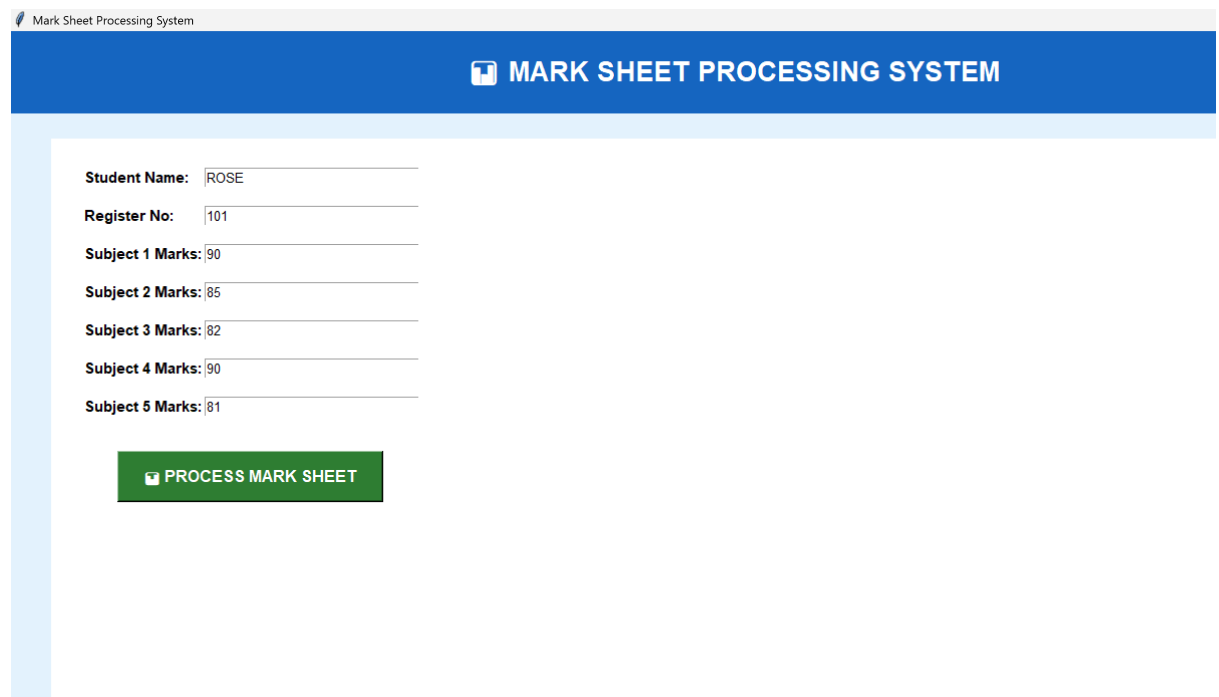
AIM

To create a simple **Mark Sheet Processing System** using Python to demonstrate **SaaS (Software as a Service)** with fields such as student name, register number, subjects, marks, average, percentage, rank, and overall performance.

PROCEDURE

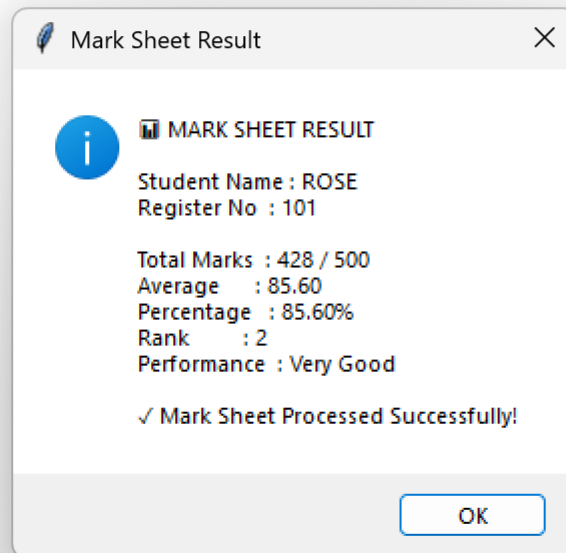
1. Open **Python IDLE** and create a new Python file.
2. Import the **Tkinter** library to create the application interface.
3. Create fields for **Student Name, Register Number, and Subject Marks**.
4. Calculate **Total, Average, Percentage, Rank, and Overall Performance**.
5. Add a **Process Mark Sheet** button to process the student details.
6. Save the program as `marksheet.py` and run it.
7. Enter the student details and click the button to display the result.

INPUT



The screenshot displays the graphical user interface of the 'Mark Sheet Processing System'. At the top, there is a title bar with the text 'Mark Sheet Processing System' and a main header bar with the title 'MARK SHEET PROCESSING SYSTEM' in white text on a blue background. Below the header, the interface features a series of input fields for student information: 'Student Name' (containing 'ROSE'), 'Register No' (containing '101'), and five 'Subject Marks' fields (containing '90', '85', '82', '90', and '81' respectively). A green button labeled 'PROCESS MARK SHEET' is positioned at the bottom of the input section. The entire form is set against a light blue background.

OUTPUT



RESULT

The **Mark Sheet Processing System** was successfully created and executed. The application accepts student marks and successfully calculates and displays the **Total, Average, Percentage, Rank, and Overall Performance**.

6. Create a simple cloud software application for Payroll Processing System using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as Name of the employee, experience in present org , over all experience, basic pay, HRA, DA, TA, net pay, gross pay etc.

AIM

To create a simple **Payroll Processing System** using Python to demonstrate **SaaS (Software as a Service)** with fields such as employee name, experience, basic pay, HRA, DA, TA, gross pay, and net pay.

PROCEDURE

1. Open **Python IDLE** and create a new Python file.
2. Import the **Tkinter** library to create the application interface.

3. Create fields for **Employee Name**, **Present Organization Experience**, **Overall Experience**, **Basic Pay**, **HRA**, **DA**, and **TA**.
4. Calculate the **Gross Pay** and **Net Pay** using the salary details.
5. Add a **Process Payroll** button to process the employee information.
6. Save the program as payroll.py and run it.
7. Enter the employee details and click **Process Payroll** to display the result.

INPUT

Payroll Processing System

ⓘ PAYROLL PROCESSING SYSTEM

Employee Name:

Experience in Present Org:

Overall Experience:

Basic Pay:

HRA:

DA:

TA:

ⓘ PROCESS PAYROLL

OUTPUT

Payroll Result

ⓘ PAYROLL RESULT

Employee Name : Riya
Present Org Exp. : 3 years
Overall Experience : 5 years

Basic Pay : ₹30000.00
HRA : ₹6000.00
DA : ₹3000.00
TA : ₹2000.00

Gross Pay : ₹41000.00
Deduction (10%) : ₹4100.00
Net Pay : ₹36900.00

✓ Payroll Processed Successfully!

OK

RESULT

The **Payroll Processing System** was successfully created and executed. The application accepts employee and salary details and successfully calculates and displays the **Gross Pay and Net Pay**.

7. Create a simple cloud software application for Students Attendance System using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as name of the students, course, present , Absent, course attendance, OD request, grade, attendance percentage, etc.

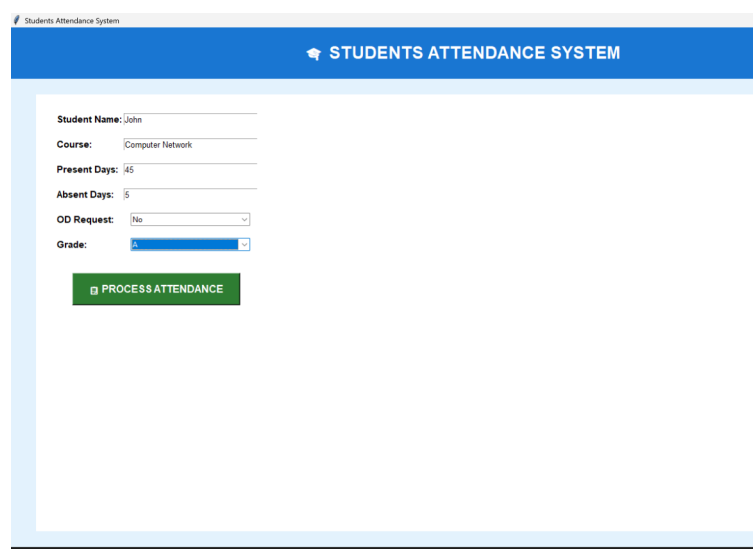
AIM

To create a simple **Students Attendance System** using Python to demonstrate **SaaS (Software as a Service)** with fields such as student name, course, present, absent, course attendance, OD request, grade, and attendance percentage.

PROCEDURE

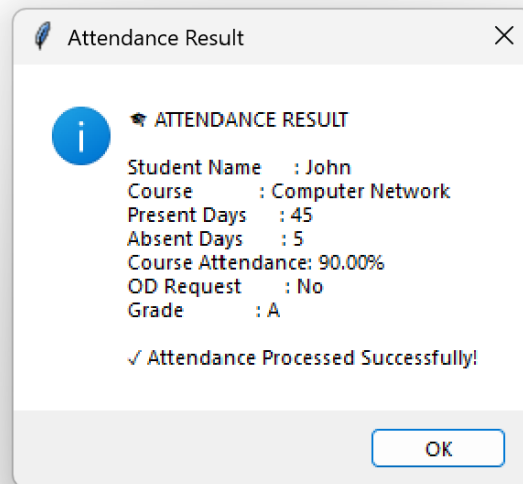
1. Open **Python IDLE** and create a new Python file.
2. Import the **Tkinter** library to create the application interface.
3. Create fields for **Student Name, Course, Present, Absent, OD Request, and Grade**.
4. Calculate the **Attendance Percentage** based on the present and absent days.
5. Add an **Attendance Process** button to process the student details.
6. Save the program as attendance.py and run it.
7. Enter the student details and click the button to display the attendance result.

INPUT



The screenshot shows a web application titled "STUDENTS ATTENDANCE SYSTEM". It features a form with the following fields: "Student Name" (text input with "John" entered), "Course" (text input with "Computer Network" entered), "Present Days" (text input with "45" entered), "Absent Days" (text input with "5" entered), "OD Request" (dropdown menu with "No" selected), and "Grade" (dropdown menu with "A" selected). Below the form is a green button labeled "PROCESS ATTENDANCE".

OUTPUT



RESULT

The **Students Attendance System** was successfully created and executed. The application accepts student attendance details and calculates and displays the **Attendance Percentage, OD Request, and Grade** successfully.